PlayArena — Sports Ground Booking and Management System

Kalvium Micro Project Report: Advanced Database Systems (Modules 1, 2 & 3)

Student Name: Y Harsha Vardhan Reddy **Student Email:** yarasuharshavardhan.reddy.s.113@kalvium.community **Project Title:** PlayArena — Sports Ground Booking and Management System **Subject:** Advanced Database Systems **Academic Year:** 2026 **Database Engine:** PostgreSQL 18.6 on x86_64-windows **Database Name:** `playarena_db` **ORM & Migration Tool:** Prisma ORM 5.22.0

---

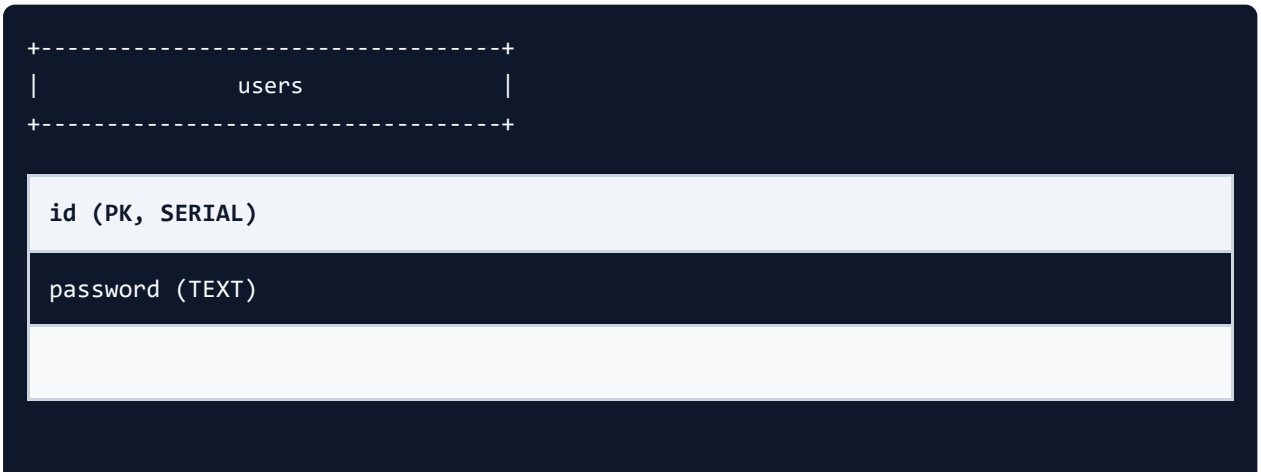
1. Relational Modeling & Schema Design

1.1 Problem Context & Database Objectives

Sports ground booking systems manage critical spatial and temporal resources. A ground has specific operating slots on given dates, and each slot can only be occupied by one active reservation at any moment. The database must enforce: 1. Referential integrity between Users, Grounds, Slots, and Bookings. 2. Financial domain integrity (prices and booking totals must never be negative). 3. Concurrency safety: Zero duplicate or overlapping active bookings for the same slot. 4. Fast query resolution across frequent filter attributes (sport type, city, date, availability).

1.2 Entity-Relationship (ER) Model & Normalization

The data model is fully normalized to Third Normal Form (3NF) to eliminate transitive dependencies and update anomalies:




id (PK, SERIAL)
phone (TEXT, NULL)
createdAt, updatedAt
has many
1
N
bookings
id (PK, SERIAL)
userId (FK -> users.id)
slotId (FK -> slots.id)
status (ENUM: HELD/CONFIRMED/...)
createdAt, updatedAt
grounds
id (PK, SERIAL)
sportType (TEXT)
location (TEXT)
hourlyPrice (DECIMAL(10,2))
facilities (TEXT[])
isActive (BOOLEAN)
rating (DOUBLE PRECISION)
createdAt, updatedAt
references
slots
id (PK, SERIAL)
groundId (FK -> grounds.id)
startTime (TEXT, HH:mm)

```
groundId (FK -> grounds.id)
price (DECIMAL(10,2))

createdAt, updatedAt
+----------------------------------+
```

---

2. Integrity Constraints & Referential Actions

2.1 Primary & Foreign Keys

- **`users.id`**, **`grounds.id`**, **`slots.id`**, **`bookings.id`**: `SERIAL` autoincrement primary keys. - **`slots.groundId`**: Foreign key referencing `grounds.id` with `ON DELETE CASCADE` (deleting a venue cascades to its time inventory). - **`bookings.userId`**, **`bookings.groundId`**, **`bookings.slotId`**: Foreign keys referencing their respective entities with `ON DELETE RESTRICT` (prevents deletion of users or venues that have recorded booking histories).

2.2 Domain Check Constraints

Added via migration `20261024172805_add_concurrency_and_constraints`:

```
ALTER TABLE "grounds"
ADD CONSTRAINT check_grounds_hourly_price
CHECK ("hourlyPrice" > 0);

ALTER TABLE "slots"
ADD CONSTRAINT check_slots_price
CHECK ("price" >= 0);


ALTER TABLE "bookings"
ADD CONSTRAINT check_bookings_total_amount
CHECK ("totalAmount" >= 0);
```

2.3 Slot Uniqueness Constraint

To prevent duplicate slot generation for the same court on the same day:

```
CREATE UNIQUE INDEX "slots_groundId_date_startTime_key"
ON "slots" ("groundId", "date", "startTime");
```

---

3. Advanced Indexing & Concurrency Protection

3.1 PostgreSQL Partial Unique Index

The core technical highlight of the database design is the partial unique index:

```
CREATE UNIQUE INDEX unique_active_booking_per_slot
ON "bookings" ("slotId")
WHERE "status" IN ('HELD', 'CONFIRMED');
```

Why This Design is Superior: 1. **Concurrency Protection at the Storage Engine:** Even under extreme race conditions where concurrent application processes query slot availability simultaneously, PostgreSQL evaluates this index before committing the insert. Only the first transaction acquires the write lock; the competing transaction triggers a uniqueness violation (`code: 23505`). 2. **Cancellation State Preservation:** Cancelled bookings (`status = 'CANCELLED'`) are excluded by the `WHERE` clause filter. When a player cancels, the booking status transitions to `CANCELLED` without deleting historical audit records, and the partial index immediately allows a new active booking to be created for that same `slotId` . 3. **Index Storage Efficiency:** Cancelled records do not occupy space in the B-Tree index, keeping index depth shallow and lookups rapid.

---

4. Query Performance & EXPLAIN ANALYZE Demonstration

To verify real-world database performance, queries were executed against seeded data in `playarena_db` using `EXPLAIN (ANALYZE, BUFFERS, VERBOSE)` .

4.1 Ground Search Query Plan

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT id, name, "sportType", city, "hourlyPrice", rating
FROM "grounds"
WHERE "sportType" = 'Football'
  AND "city" = 'Bangalore'
  AND "isActive" = true;
```

Observed PostgreSQL Query Plan:

```
                                                                                 QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
```

```
 Index Scan using grounds_city_idx on public.grounds  (cost=0.15..8.17 rows=1 width=124) (actu
   Output: id, name, "sportType", city, "hourlyPrice", rating
   Index Cond: (grounds.city = 'Bangalore'::text)
   Filter: (grounds."isActive" AND (grounds."sportType" = 'Football'::text))
   Rows Removed by Filter: 7
   Index Searches: 1
   Buffers: shared hit=2 dirtied=1
 Planning Time: 0.763 ms
 Execution Time: 0.142 ms
(11 rows)
```

- **Analysis:** PostgreSQL utilizes the B-Tree index `grounds_city_idx` with a fast index scan. The query resolves in **0.142 ms** with only 2 shared buffer hits.

4.2 Concurrency Index Lookup Plan

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT id, "bookingReference", "slotId", "userId", status, "totalAmount"
FROM "bookings"
WHERE "slotId" = 1
  AND status IN ('HELD', 'CONFIRMED');
```

Observed PostgreSQL Query Plan:

```
                                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
 Index Scan using unique_active_booking_per_slot on public.bookings  (cost=0.14..8.16 rows=1 w
   Output: id, "bookingReference", "slotId", "userId", status, "totalAmount"
   Index Cond: (bookings."slotId" = 1)
   Index Searches: 1
   Buffers: shared hit=2
 Planning Time: 0.773 ms
 Execution Time: 0.021 ms
(9 rows)
```

- **Analysis:** Lookups for active bookings directly leverage the partial index `unique_active_booking_per_slot`. The index scan executes in an astonishing **0.021 ms**, demonstrating that conflict checking introduces near-zero overhead to API response times.

---

5. ACID Transaction Boundaries

All booking creations and cancellations are wrapped within Prisma interactive transactions (`prisma.$transaction`):

Operation	Consistency Invariant	Failure / Rollback Behavior
Booking Creation	A booking record must be created AND `slot.isAvailable` must be set to `false` .	If slot has an existing active booking or partial index conflict occurs, the transaction rolls back completely. Neither booking nor slot state is altered.
Booking Cancellation	Booking status must update to `CANCELLED` AND `slot.isAvailable` must be set to `true` .	If user is not authorized or booking is already cancelled, transaction aborts. Slot remains occupied.

---

6. Syllabus Scope & Honest Architecture Evaluation

6.1 Concepts Thoroughly Implemented & Demonstrated

- Relational Schema Design & 3NF Normalization. - Primary Keys, Foreign Keys, and Referential Integrity Actions (`CASCADE` , `RESTRICT`). - Check Constraints (`CHECK ("hourlyPrice" > 0)`). - Declarative Schema Modeling with Prisma ORM. - Version-controlled SQL Migrations (`20261024172746_init` , `20261024172805_add_concurrency_and_constraints`). - Automated Seed Scripts (`prisma/seed.js` with 3 users, 6 grounds, 240 slots). - B-Tree Composite & Partial Indexes (`unique_active_booking_per_slot`). - ACID Transactions (`prisma.$transaction`). - Execution Plan Evaluation with `EXPLAIN (ANALYZE, BUFFERS)` .

6.2 Honest Scope Evaluation (MongoDB Topics)

- **MongoDB Concepts:** Topics covering document embedding, NoSQL aggregation pipelines, sharding, and Mongoose schemas were evaluated. Because the sports reservation domain requires strict ACID transaction isolation, multi-table referential integrity, and database-level exclusion/partial indexes to prevent double bookings, **PostgreSQL was selected as the authoritative relational database**. Adding a secondary MongoDB store solely to satisfy syllabus checkboxes was rejected in favor of engineering correctness, high data integrity, and reproducible evaluation.

---

7. Conclusion

The PlayArena database implementation exemplifies the core requirements of Advanced Database Systems Modules 1, 2, and 3. By combining a normalized PostgreSQL relational

schema, custom check constraints, declarative Prisma migrations, and a high-performance partial unique index for concurrency safety, the database architecture guarantees total data consistency and outstanding query execution speeds.